

Network Programming 2026 - Assignment #1

Course: Network Programming

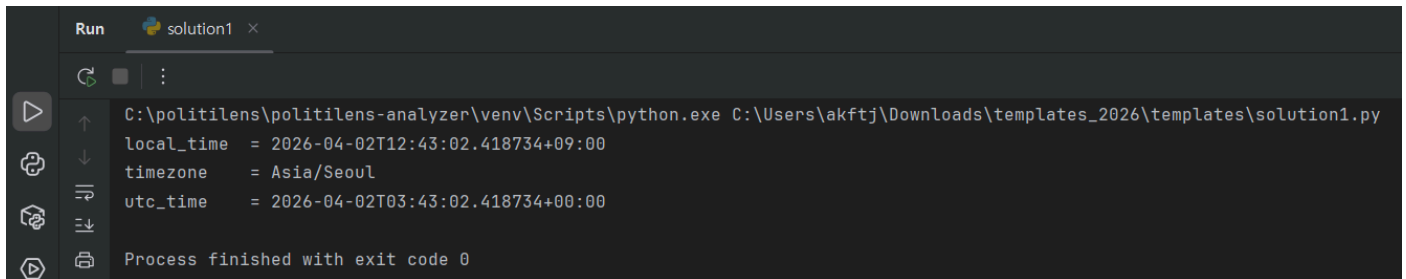
Assignment: Assignment #1 (Chapters 1, 2, 3)

Name: 홍성주

Student ID: 20221528

Question 1. Raw Network Conversation

1. Screenshots



```
Run solution1 x
C:\politilens\politilens-analyzer\venv\Scripts\python.exe C:\Users\akftj\Downloads\templates_2026\templates\solution1.py
local_time = 2026-04-02T12:43:02.418734+09:00
timezone = Asia/Seoul
utc_time = 2026-04-02T03:43:02.418734+00:00
Process finished with exit code 0
```

2. My Code Logic

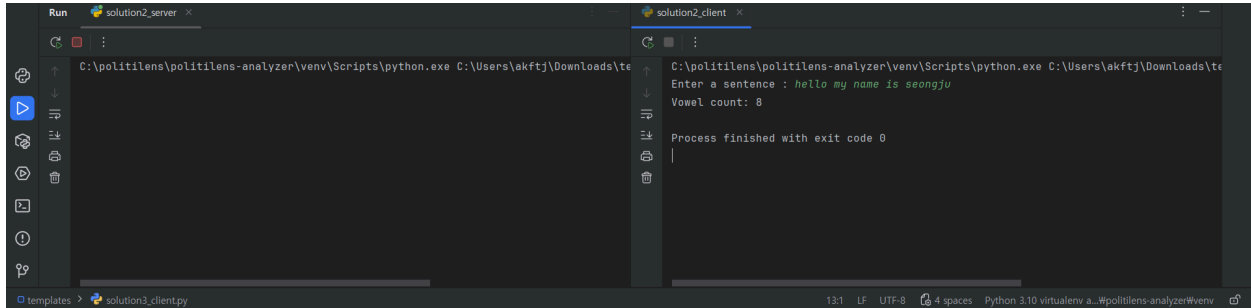
For this question, I didn't use any external HTTP libraries like requests. I only used the built-in socket and json modules in Python. Because the original worldtimeapi.org server was down, I followed the TA's announcement and used timeapi.io (port 443) instead. Since port 443 needs HTTPS, I used the ssl module to wrap my TCP socket. I manually created the HTTP GET request header with \r\n and sent it to the server. After receiving the response, I split the header and the JSON body. Finally, I parsed the JSON to print out local_time, timezone, and utc_time as required.

3. How I Handled Errors

- I used try-except blocks to handle possible errors.
- socket.error: Catch network problems like connection failures.
- json.JSONDecodeError: Catch errors if the server sends bad JSON data.
- Exception: Catch any other unexpected errors so the program doesn't crash suddenly.

Question 2. UDP Vowel Counter Service

1. Screenshots here



2. My Code Logic

I used `socket.SOCK_DGRAM` to make a UDP server and client.

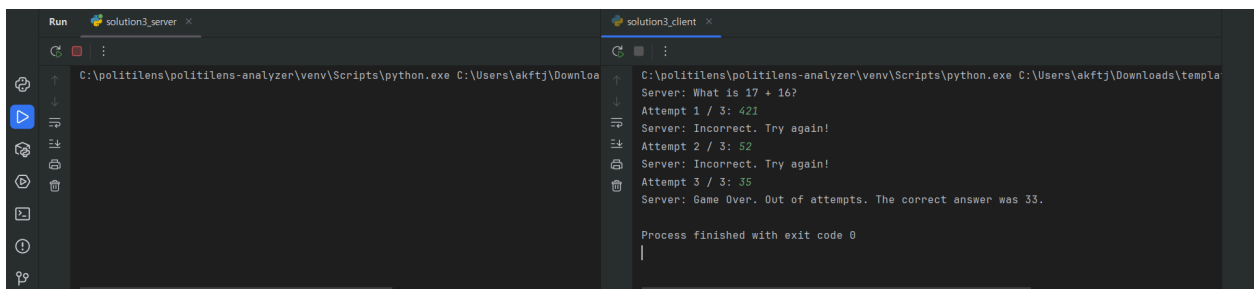
- **Server:** I used a `while True:` loop so the server doesn't stop and can handle multiple clients one by one. When it receives a message, it checks every character to see if it is a vowel (both uppercase and lowercase). Then, it sends back a message like "Vowel count: X".
- **Client:** I added a `settimeout()` function to the socket. This prevents the client from waiting forever if the server doesn't reply. It simply takes the user's input, sends it, and prints the server's reply.

3. How I Handled Errors

- I added `try-except socket.error` in both the server and client codes to handle any socket connection issues.
- I also used a `finally` block to call `close()`. This makes sure the socket closes safely even if an error occurs.

Question 3. TCP Math Quiz Game

1. Screenshots



```
Run solution3_server x
C:\politilens\politilens-analyzer\venv\Scripts\python.exe C:\Users\akftj\Downloa

solution3_client x
C:\politilens\politilens-analyzer\venv\Scripts\python.exe C:\Users\akftj\Downloads\temple
Server: What is 7 + 8?
Attempt 1 / 3: hello
please enter an integer
Attempt 1 / 3: 7
Server: Incorrect. Try again!
Attempt 2 / 3: 15
Server: Correct! You win.
Process finished with exit code 0
```

2. My Code Logic

I used `socket.SOCK_STREAM` for reliable TCP communication.

- **Server:** The server waits for clients. When a client connects, it calls the `play_game` function. First, it checks if the client sent the word "start". If yes, it creates two random numbers from 1 to 20 and sends a math question.
- **Client:** The client uses a `for` loop to give the user exactly 3 chances. If the user gets the right answer ("Correct") or runs out of chances ("Game Over"), the loop breaks and the game ends.

3. How I Handled Errors

Input validation was the most important part to prevent server crashes.

- **Client-side:** I used a `while True` loop and `try-except ValueError` in the `get_integer_answer` function. If the user types a string instead of a number, it asks them to type an integer again.
- **Server-side:** I also put `try-except ValueError` in the server logic. Just in case the server receives something that is not an integer, it sets `ans = None`. This completely stops the server from crashing.